

Capitolo 30 — PROT-001 — Git & Working Tree Safety

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-30
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/30_prot_001_git_working_tree_safety.md
Source SHA	9dc5ee3
Release	2026-09-04T09:44:50.631Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

30.0 Prima di cambiare qualcosa, capire che cosa c'è già

Quando più persone o servizi lavorano sugli stessi materiali, uno dei rischi più semplici è anche uno dei più costosi: iniziare un intervento senza accorgersi che nello spazio di lavoro esistono già modifiche non comprese.

PROT-001 — Git & Working Tree Safety governa questo rischio nelle operazioni Git del WCM.

Git è uno strumento che conserva la storia delle modifiche ai file. Il **working tree** è, in termini semplici, l'insieme dei file su cui si sta lavorando in quel momento. Può coincidere con l'ultima versione registrata oppure contenere modifiche ancora non consolidate.

Il protocollo parte da una regola prudenziale:

una modifica locale che non è stata compresa non va cancellata, sovrascritta o aggirata per comodità.

Questa prudenza non significa però che ogni segnale di modifica debba bloccare il lavoro. Un file può apparire modificato anche per ragioni tecniche che non corrispondono a un cambiamento sostanziale del suo contenuto. Per questo il protocollo impone di **verificare**, non semplicemente di reagire al primo indicatore.

30.1 Il problema che PROT-001 risolve

Immaginiamo un caso pedagogico.

Un operatore deve aggiornare un documento. Prima di iniziare scopre che il sistema segnala quel file come già modificato.

Esistono almeno due possibilità molto diverse:

1. qualcuno ha realmente cambiato il contenuto e quel lavoro deve essere preservato;
2. il sistema segnala una differenza tecnica o di normalizzazione che non rappresenta una modifica sostanziale.

Trattare entrambe le situazioni nello stesso modo sarebbe un errore.

Se si ignorasse sempre il segnale, si potrebbe perdere lavoro reale. Se invece ogni segnale producesse automaticamente un blocco, si potrebbero fermare attività perfettamente sicure.

PROT-001 introduce quindi un gate iniziale che serve a distinguere un rischio reale da un'apparenza di rischio.

30.2 Quando si attiva

Il protocollo si applica nell'ambito delle operazioni Git del WCM, in particolare prima di modificare file o sincronizzare il repository tramite **pull**.

Il trigger pratico è quindi l'avvio di un'attività che può cambiare o sincronizzare contenuti versionati.

Prima dell'azione occorre conoscere almeno:

- il repository su cui si sta lavorando;
- il branch autorizzato;
- lo stato corrente del working tree;
- lo scope autorizzato del lavoro, compresi i path sui quali è consentito operare.

Il protocollo non assegna da solo questi permessi. Li verifica e li rispetta nel contesto dell'authority già esistente.

30.3 Il gate iniziale

La sequenza canonica parte da tre controlli:

```
VERIFICA REPOSITORY E BRANCH
    ↓
VERIFICA STATO DEL WORKING TREE
    ↓
CI SONO MODIFICHE SEGNALATE?
├─ NO → PASS
└─ SÌ → VERIFICA LA NATURA DELLA DIFFERENZA
```

Nel file tecnico il controllo dello stato è espresso con **git status**. Non è necessario conoscere il comando per capire il principio: prima di scrivere, il sistema deve sapere se lo spazio di lavoro è pulito oppure contiene differenze rispetto alla baseline registrata.

Se esistono differenze, il protocollo richiama **PLAY-001** prima di concludere che esista un vero blocco. Il punto importante, sul piano concettuale, è che **segnalato come modificato** e **materialmente modificato** non sono sinonimi.

30.4 Vedere una “M” non basta

In Git un file modificato può essere indicato con una **M**. PROT-001 vieta di interpretare automaticamente quel simbolo come prova sufficiente di una modifica sostanziale.

Occorre verificare il **diff**, cioè il confronto tra ciò che il file contiene ora e ciò che conteneva nella baseline di riferimento.

In linguaggio semplice:

SEGNALE DI DIFFERENZA

≠

PROVA DEL SIGNIFICATO DELLA DIFFERENZA

Questo passaggio è importante perché alcuni cambiamenti tecnici — per esempio una diversa normalizzazione dei caratteri di fine riga — possono far apparire un file modificato senza che una persona abbia davvero cambiato il suo significato.

Il protocollo deriva anche da evidenze operative nelle quali questo tipo di falso positivo è stato osservato. Tale evidenza sostiene la baseline corrente, ma non implica che ogni possibile ambiente Git sia già stato validato sul campo.

30.5 Il decision point: continuare o bloccare

Dopo la verifica, il gate distingue due casi.

Caso A — nessuna modifica sostanziale non autorizzata

Se la differenza è riconducibile a normalizzazione o metadati e la verifica conferma che non esiste lavoro reale da proteggere, il controllo può essere aggiornato e l'attività può continuare.

Caso B — modifica reale non autorizzata o non compresa

Se il working tree contiene invece una modifica sostanziale che non rientra nel lavoro autorizzato o che non è stata compresa, il protocollo porta a **BLOCKED**.

Questo blocco non è una punizione e non è una valutazione sulla qualità della modifica. Significa semplicemente che il sistema non possiede condizioni sufficientemente sicure per procedere senza rischiare di alterare lavoro esistente.

30.6 Le operazioni distruttive non sono una scorciatoia

Quando una modifica locale impedisce di proseguire, può essere tecnicamente possibile cancellarla o riscrivere la storia per “ripulire” la situazione.

PROT-001 vieta di usare questa possibilità come scorciatoia.

In particolare, operazioni come **reset --hard**, force push, rebase o merge correttivi richiedono autorizzazione esplicita.

Per un lettore non tecnico, la distinzione essenziale è questa:

- alcune operazioni aggiungono lavoro mantenendo leggibile la storia;
- altre possono cancellare modifiche, riscrivere la storia o alterare il punto di riferimento condiviso.

Il protocollo impedisce che la seconda categoria venga usata autonomamente per risolvere un ostacolo operativo.

30.7 Sincronizzare senza riscrivere la storia

Anche la sincronizzazione con il repository remoto è vincolata.

Nel pre-sync, `git pull` deve essere eseguito con l'opzione `--ff-only`.

In termini semplici, il sistema accetta l'avanzamento quando la storia locale può essere portata avanti senza creare automaticamente una riconciliazione più complessa. Se ciò non è possibile, l'operazione non deve inventare una soluzione implicita.

Questo riflette una caratteristica più generale del WCM: quando la situazione richiede una decisione che supera il perimetro meccanico autorizzato, il sistema non deve mascherarla come semplice routine tecnica.

30.8 Commit e push: anche una scrittura corretta può essere fuori scope

Avere un working tree sicuro non autorizza a modificare qualsiasi cosa.

PROT-001 stabilisce che commit e push siano consentiti soltanto sul branch e sui path autorizzati dal Service Job.

Prima del commit devono essere verificati:

- lo scope effettivo delle modifiche;
- il diff;
- quando pertinente, la pulizia formale del diff tramite `git diff --check`.

Il principio è semplice: **sicurezza tecnica e authority sono due controlli distinti**.

Un'operazione può essere tecnicamente sicura ma non autorizzata. Oppure può essere autorizzata nello scopo ma tecnicamente rischiosa nelle condizioni correnti. Per procedere servono entrambe le condizioni.

30.9 L'identità Git non è l'identità organizzativa

Un commit registra un autore tecnico. Questo dato serve a tracciare chi o quale identità Git ha prodotto quella registrazione.

Nel WCM, però, questa identità non assegna automaticamente ruolo, authority o ownership organizzativa.

In altre parole:

```
AUTORE GIT
≠
AUTHORITY WCM
```

La distinzione evita un errore concettuale importante: poter materialmente effettuare una scrittura non significa avere il diritto organizzativo di decidere che quella scrittura debba avvenire.

Per la stessa ragione, una configurazione Git repository-local già valida non deve essere modificata senza necessità.

30.10 Output del protocollo

L'output principale di PROT-001 non è un nuovo documento: è un esito operativo affidabile del gate.

Il risultato può essere sintetizzato così:

```
WORKING TREE PULITO
→ PASS

WORKING TREE MODIFICATO
→ VERIFICA
→ SOLO NORMALIZZAZIONE / METADATI
  → REFRESH / RE-CHECK
  → CONTINUA

WORKING TREE MODIFICATO
→ VERIFICA
→ MODIFICA REALE NON AUTORIZZATA O NON COMPRESA
  → BLOCKED
```

Quando il gate passa, l'attività può proseguire entro branch, path e authority già autorizzati. Quando il gate blocca, il protocollo impedisce che il problema venga “risolto” cancellando o sovrascrivendo ciò che non si è ancora compreso.

30.11 Failure mode

Il protocollo esiste perché diversi errori apparentemente piccoli possono produrre conseguenze rilevanti.

I principali failure mode sono:

- iniziare a scrivere senza controllare repository e branch;
- confondere un semplice indicatore di modifica con la prova di un conflitto reale;
- ignorare una modifica sostanziale già presente;
- cancellare o sovrascrivere lavoro locale non compreso;
- usare operazioni Git distruttive per aggirare un blocco;
- sincronizzare creando implicitamente una riconciliazione non autorizzata;
- fare commit o push fuori dai path o dal branch autorizzati;

- confondere l'identità tecnica del commit con l'autorità organizzativa.

Il failure mode più profondo è procedere sulla base di un'assunzione invece che di una verifica.

30.12 Relazioni con il resto del WCM

PROT-001 è un protocollo trasversale di sicurezza per le operazioni Git. Non sostituisce il processo che descrive il lavoro da svolgere e non crea un nuovo workflow.

Interviene come vincolo quando un processo o un'attività autorizzata richiede operazioni sul repository.

La sua logica è coerente con altri principi WCM già presenti nella baseline: lavorare entro authority e scope, evitare inferenze quando esiste un controllo diretto disponibile e non trasformare un limite o un'anomalia in una scorciatoia distruttiva.

Queste relazioni aiutano a leggere PROT-001 nel sistema, ma non estendono il protocollo oltre il suo ambito canonico: **operazioni Git nel WCM**.

30.13 Maturity e limiti

La baseline canonica classifica PROT-001 come **VALIDATED**.

Le sue evidenze derivano da prove operative del Service Bridge e includono prerequisiti reali relativi a branch, fetch, identità Git e working tree, oltre a un episodio di falsa modifica dovuta a normalizzazione LF/CRLF.

Questa qualifica va interpretata nel perimetro delle evidenze disponibili. Non significa che ogni combinazione di sistema operativo, configurazione Git, hosting, client o scenario collaborativo sia stata universalmente validata.

Il protocollo inoltre non decide il significato di una modifica, non amplia l'autorità e non autorizza operazioni distruttive. Stabilisce il comportamento sicuro da adottare quando il lavoro passa attraverso Git.

30.14 La regola da ricordare

Se di questo capitolo dovesse restare una sola idea, è questa:

prima di modificare uno spazio di lavoro condiviso, verifica ciò che c'è già; se una differenza non è compresa, non cancellarla per poter continuare.

PROT-001 trasforma questa prudenza in una regola operativa: controllare, distinguere il segnale dal rischio reale, rispettare branch e scope, e fermarsi quando procedere significherebbe sovrascrivere qualcosa che il sistema non ha ancora compreso.

Source map

Fonti canoniche utilizzate per il Technical Truth Pass:

- [wcm/documentation/process-memory-book/BOOK_INDEX.md](#) — mapping editoriale CH30 → PROT-001;
- [wcm/process-book/protocols/PROT-001_GIT_WORKTREE_SAFETY.md](#) — baseline tecnica primaria del protocollo;
- [WCM_AGENT_START.md](#) — principi correnti di authority, scope e comportamento fail-safe richiamati solo dove pertinenti.

Maturity qualifier: [PROT-001](#) è [VALIDATED](#) nella baseline corrente; le evidenze dichiarate sono circoscritte ai POC e agli episodi operativi citati dal protocollo canonico e non costituiscono una validazione universale di ogni ambiente Git.